



ASP.NET Core Identity + JWT

Authentication & Authorization — Step by Step Guide



Overview

ASP.NET Core Identity হল একটি membership system যা user registration, login, role management করে। JWT (JSON Web Token) দিয়ে stateless authentication implement করা হয় — মানে server কোনো session রাখে না, প্রতিটি request এর সাথে token পাঠাতে হয়।



Required NuGet Packages

Package	Purpose
Microsoft.AspNetCore.Identity.EntityFrameworkCore	Identity + EF Core integration
Microsoft.EntityFrameworkCore.SqlServer	SQL Server database provider
Microsoft.EntityFrameworkCore.Tools	Migration commands (PMC)
Microsoft.AspNetCore.Authentication.JwtBearer	JWT token validation middleware
System.IdentityModel.Tokens.Jwt	JWT token generation



Install via: Tools → NuGet Package Manager → Package Manager Console → Install-Package <name>



Step 1 — appsettings.json Configuration

Connection string এবং JWT settings — appsettings.json এ রাখো:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\mssqllocaldb;
      Database=AppIdentityDb;Trusted_Connection=True;
      TrustServerCertificate=True"
  },
  "JwtSettings": {
    "Key": "YourSuperSecretKey_AtLeast32Characters!!",
    "Issuer": "https://localhost:5001",
```

```

    "Audience": "https://localhost:5001",
    "ExpireMinutes": 60
  }
}

```

Key	কী কাজ করে
Key	JWT sign করার জন্য secret key — minimum 32 characters হতে হবে
Issuer	Token কে issue করেছে — সাধারণত তোমার server URL
Audience	Token কে use করবে — client URL
ExpireMinutes	Token কতক্ষণ valid থাকবে (এখানে 60 মিনিট)

🔗 Step 2 — ApplicationDbContext তৈরি করো

DbContext টা IdentityDbContext থেকে inherit করতে হবে — এটাই Identity এর tables তৈরি করবে:

```

using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options) { }
}

```

⚠ Important: IdentityDbContext<ApplicationUser> inherit করলে EF Core নিজেই AspNetUsers, AspNetRoles, AspNetUserRoles সহ সব Identity table তৈরি করবে।

🔗 Step 3 — ApplicationUser Model (Optional Custom Fields)

Default Identity User এর সাথে extra fields যোগ করতে চাইলে:

```

using Microsoft.AspNetCore.Identity;

public class ApplicationUser : IdentityUser
{
    public string FullName { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}

```

 **Note:** Custom fields না লাগলে IdentityUser সরাসরি use করতে পারো। তাহলে ApplicationDbContext : IdentityDbContext লিখলেই হবে (generic type ছাড়া)।

Step 4 — Program.cs এ সব Register করো

DI Container এ DbContext, Identity এবং JWT Authentication register করতে হবে:

```
var builder = WebApplication.CreateBuilder(args);

// — 1. Connection String —————
var connectionString = builder.Configuration
    .GetConnectionString("DefaultConnection");

// — 2. DbContext Register —————
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(connectionString));

// — 3. Identity Register —————
builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options => {
    options.Password.RequireDigit = true;
    options.Password.RequiredLength = 6;
    options.Password.RequireUppercase = false;
})
.AddEntityFrameworkStores<AppDbContext>()
.AddDefaultTokenProviders();

// — 4. JWT Settings Read —————
var jwtSettings = builder.Configuration.GetSection("JwtSettings");
var key = Encoding.UTF8.GetBytes(jwtSettings["Key"]!);

// — 5. JWT Authentication Register —————
builder.Services.AddAuthentication(options => {
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options => {
    options.TokenValidationParameters = new TokenValidationParameters {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
```

```

        ValidateIssuerSigningKey = true,
        ValidIssuer                = jwtSettings["Issuer"],
        ValidAudience             = jwtSettings["Audience"],
        IssuerSigningKey           = new SymmetricSecurityKey(key)
    };
});

builder.Services.AddControllers();

var app = builder.Build();

app.UseAuthentication(); // ← Identity/JWT check
app.UseAuthorization();  // ← Role/Policy check
app.MapControllers();
app.Run();

```

🔗 Step 5 — Migration চালাও

Identity সব table নিজেই তৈরি করবে — শুধু migration run করতে হবে:

```

// Package Manager Console এ:
Add-Migration Initial
Update-Database

// এতে নিচের tables তৈরি হবে:
//AspNetUsers, AspNetRoles, AspNetUserRoles,
// AspNetUserClaims, AspNetRoleClaims, AspNetUserTokens,
// AspNetUserLogins

```

🔗 Step 6 — AuthController তৈরি করো

6A. Register Endpoint

```

[HttpPost("register")]
public async Task<IActionResult> Register(RegisterDto model)
{
    var user = new ApplicationUser {
        UserName = model.Email,
        Email    = model.Email,
        FullName = model.FullName
    };
}

```

```

};

var result = await _userManager.CreateAsync(user, model.Password);

if (!result.Succeeded)
    return BadRequest(result.Errors);

return Ok(new { message = "User registered successfully" });
}

```

6B. Login Endpoint + JWT Token Generate

```

[HttpPost("login")]
public async Task<IActionResult> Login(LoginDto model)
{
    var user = await _userManager.FindByEmailAsync(model.Email);
    if (user == null) return Unauthorized();

    var isValid = await _userManager.CheckPasswordAsync(user, model.Password);
    if (!isValid) return Unauthorized();

    // Token Generate
    var token = GenerateJwtToken(user);
    return Ok(new { token });
}

private string GenerateJwtToken(ApplicationUser user)
{
    var jwtSettings = _config.GetSection("JwtSettings");
    var key = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(jwtSettings["Key"]!));

    var claims = new[] {
        new Claim(JwtRegisteredClaimNames.Sub, user.Id),
        new Claim(JwtRegisteredClaimNames.Email, user.Email!),
        new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
    };

    var token = new JwtSecurityToken(
        issuer: jwtSettings["Issuer"],
        audience: jwtSettings["Audience"],
        claims: claims,

```

```

        expires: DateTime.UtcNow.AddMinutes(
            int.Parse(jwtSettings["ExpireMinutes"]!)),
        signingCredentials: new SigningCredentials(
            key, SecurityAlgorithms.HmacSha256)
    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}

```

Step 7 — Protected Endpoint বানাও

[Authorize] attribute দিলে সেই endpoint শুধুমাত্র valid JWT token দিয়ে access করা যাবে:

```

[Authorize]
[HttpGet("profile")]
public IActionResult GetProfile()
{
    var userId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
    var email = User.FindFirst(ClaimTypes.Email)?.Value;
    return Ok(new { userId, email });
}

// Role-based authorization:
[Authorize(Roles = "Admin")]
[HttpGet("admin-only")]
public IActionResult AdminPanel() => Ok("Admin access granted!");

```

JWT কীভাবে কাজ করে? (Flow)

#	ধাপ	কী হয়
1	Client → POST /login	Email ও Password পাঠায়
2	Server validates	UserManager দিয়ে user ও password check করে
3	Server generates JWT	Claims সহ token তৈরি করে secret key দিয়ে sign করে
4	Client receives token	Token local storage বা cookie তে save করে
5	Client → GET /profile	Authorization: Bearer <token> header পাঠায়
6	Server validates token	JwtBearer middleware token verify করে
7	Response	Valid হলে data return করে, invalid হলে 401 দেয়

DTO Classes

```
// RegisterDto.cs
public class RegisterDto {
    public string FullName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}

// LoginDto.cs
public class LoginDto {
    public string Email { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}
```

Quick Reference — Important Commands

Command / Code	কাজ
Add-Migration Initial	Identity সহ প্রথম migration তৈরি করে
Update-Database	Database এ migration apply করে সব table তৈরি করে
[Authorize]	Endpoint টি JWT token ছাড়া access হবে না
[AllowAnonymous]	Login/Register এর মতো public endpoint এ দাও
UserManager.CreateAsync()	নতুন user তৈরি করে password hash সহ
SignInManager.CheckPasswordAsync()	Password সঠিক কিনা check করে
User.FindFirst(ClaimTypes.Email)	Controller এ current user এর info পাও

 **মনে রেখো:** app.UseAuthentication() অবশ্যই app.UseAuthorization() এর আগে লিখতে হবে, না হলে JWT validation কাজ করবে না।